CS 391
Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

Week - 10

Outline

1. Lifecycle of Components
2. Hooks

Lifecycle of Components

1. Lifecycle of Components
2. Hooks

Lifecycle of Components…

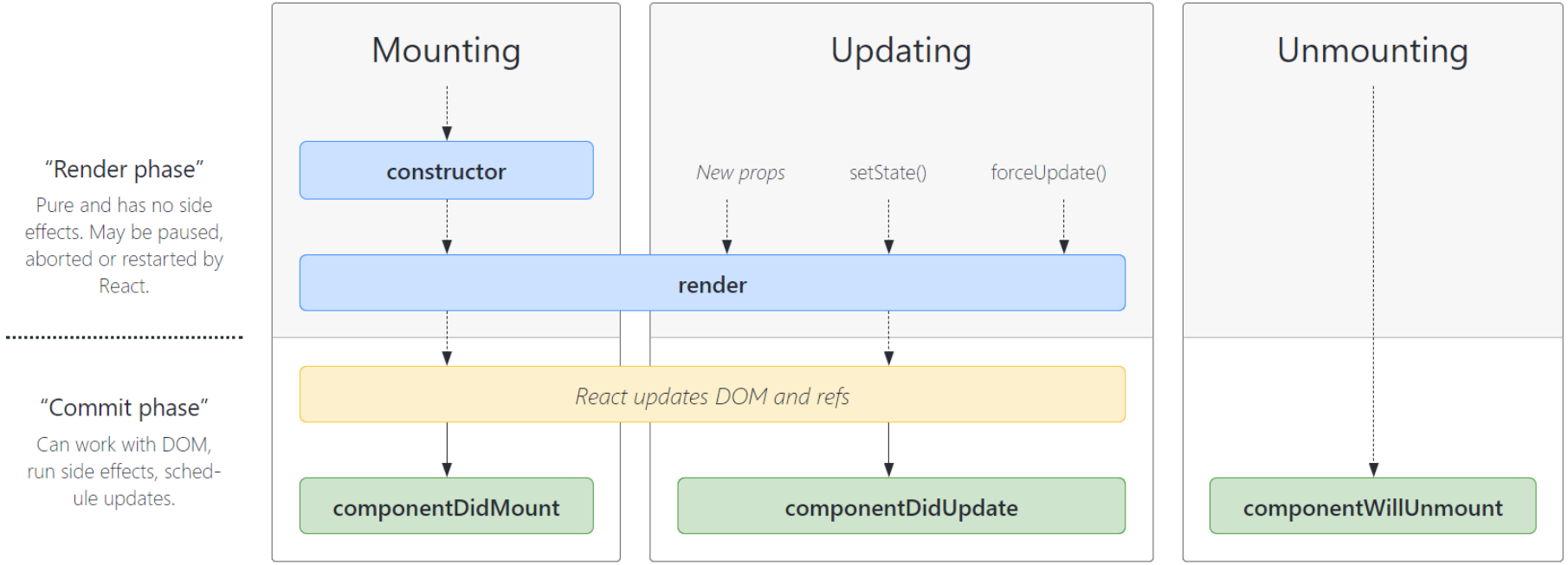
- Each component in React has a lifecycle which you can monitor and manipulate during its three main phases:
  - Mounting
  - Updating
  - Unmounting

https://retool.com/blog/the-react-lifecycle-methods-and-hooks-explained/

Lifecycle of Components

- In a ***class-based*** component, you can call different methods for each phase of the lifecycle

https://projects.wojtekmaj.pl/react-lifecycle-methods-diagram/



These lifecycle methods are of course not applicable to functional components because they can only be written/contained within a class. However, React hooks give functional components the ability to use states

“mounting” phase

- The mounting phase is when a new component is created and inserted into the DOM or, in other words, when the life of a component begins.
- This can only happen **once** and is often called “initial render.”

“updating” phase

- The updating phase is when the component updates or re-renders.
- This reaction is triggered when the props are updated or when the state is updated.
- This phase can occur multiple times, which is kind of the point of React.

“unmounting” phase

- The last phase within a component's lifecycle is the unmounting phase, when the component is removed from the DOM.

What are React Hooks?

1. Lifecycle of Components
2. Hooks

What are React Hooks?

- Hooks are a new addition in React 16.8.
- Functions starting with **`use`** are called *Hooks*.
- They let you use state and other React features without writing a class
  - Hooks don't work inside classes — they let you use React without classes.
  - Hooks are functions that let you "hook into" React state and lifecycle features from function components.
- They also work together with existing code, so they can easily be adopted into a codebase.

No Breaking Changes

- Completely opt-in.
  - You can try Hooks in a few components without rewriting any existing code.
  - But you don’t have to learn or use Hooks right now if you don’t want to.
- 100% backwards-compatible.
  - Hooks don’t contain any breaking changes.
- There are no plans to remove classes from React.
- Hooks don’t replace your knowledge of React concepts.
  - Hooks provide a more direct API to the React concepts you already know
  - Hooks also offer a new powerful way to combine them.

Hooks API

React provides a few built-in Hooks like useState.
You can also create your own Hooks to reuse stateful behavior between different components

- Basic Hooks
  - useState
  - useEffect
  - useContext
  - useRouter
- Additional Hooks
  - useReducer
  - useCallback
  - useMemo
  - useRef
  - useLayoutEffect

useState

- Syntax:
  - const [state, setState] = useState(initialState);
- useState returns a pair:
  - the **current state** value
  - **a function** that lets you update it.
- You can call this function from an event handler or somewhere else.

Simple Example…

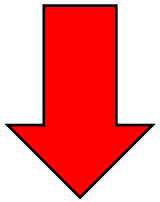
first "Hook"

```
import React, { useState } from 'react';

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

1. count is equal to this.state.count
2. setCount is equal to this.setState({ count: 0 })
3. useState is equal to constructor state.
   0 is the default state value

useState

```
const [count, setCount] = useState(123);
```



meaning

```
var countVariable = useState(123); // Returns a pair
var count =  countVariable[0]; // First item in a pair
var setCount =  countVariable[1]; // Second item in a pair
```

Declaring multiple state variables

```
const [grade, setGrade] = useState(89);
const [name,  setName]  = useState('Berke');
const [todos, setTodos] = useState([ {text: 'Learn Hooks'},
                                     {text: 'Study Math'}]);
```

useRouter…

- The useRouter hook allow functional components to **access and manipulate the router object**.
- It enables you to read route information, navigate programmatically, and handle dynamic routes.

useRouter Key Features

- Access the current route:
  - **router.pathname** → The current route path
  - **router.query** → Route parameters for dynamic routes
- Programmatic navigation:
  - **router.push**('/path') → Navigate to a new route
  - **router.replace**('/path') → Navigate without adding a new entry to the browser history
- Other router methods and properties:
  - **router.back**() → Go back in history
  - **router.reload**() → Reload the current page
  - **router.events** → Subscribe to route change events

Example of useRouter

```
import { useRouter } from 'next/router';
function ProductDetail() {
  const router = useRouter();
  const { productId } = router.query;
  const goHome = () => {
    router.push('/');
  };
return (...)
```

dynamic route param

navigate to home page

```
return (
  <div>
    <h1>Product ID: {productId}</h1>
    <button onClick={goHome}>Go Home</button>
  </div>
 );
}
export default ProductDetail;
```

Effect Hook, useEffect

- The Effect Hook in ReactJS is used to handle the side effects
- "side effects" refer to any operations or behaviors that occur in a component after rendering, and that don't directly impact the current component render cycle. These side effects can include tasks such as
  - Fetching data from an API.
  - Setting up event listeners or subscriptions.
  - Manipulating the DOM directly (although React generally handles DOM manipulation for you).
  - Cleaning up resources when a component unmounts.
  - Interactions with the outside world.
- Effect Hook serves the same purpose as *componentDidMount*, *componentDidUpdate*, and *componentWillUnmount* in React classes, but unified into a single API

useEffect

- By using this Hook, ***you tell React that your component needs to do something after render.***
- **Syntax:**
  - accepts two arguments.
    The second argument is optional.
  - useEffect(<function>, <dependency>)

```
useEffect(() => {
    // Side effect logic goes here
    return () => {
        // Cleanup logic (optional)
  };
}, [dependencies]);
```

This optional return function cleans up side effects like subscriptions or timers when the component unmounts

- The function passed to *useEffect will run after the render is committed* to the screen

useEffect

```
import React, { useState, useEffect } from 'react';
function Example() {
   const [count, setCount] = useState(0);
   useEffect(() => {
       document.title = `You clicked ${count} times`; //Update the document title using
                                                      //the browser API
   }
   )
   return (
       <div>
          <p>You clicked {count} times</p>
          <button onClick={() => setCount(count + 1)}>Click me</button>
       </div>
);
}
```

Conditionally firing an effect…

- The default behavior for effects is to fire the effect after **every completed render.**
- You can tell React to *skip* applying an effect if certain values haven't changed between re-renders
  - by adding a second parameter which is an array that contains a list of state variables to watch for.

```
useEffect(
  () => {
    console.log(`Hi ${name} you clicked ${count} times`)
  },
  [name, count]
)
```

React will only re-run the side effect if one of the items changes.

…Conditionally firing an effect

- Similarly, you can tell React to only execute the side effect once (at mount time), by passing an empty array:

```
useEffect(
  () => {
    console.log(`Component mounted`)
  },
  []
)
```

Effect Cleanup

- Some effects require cleanup to reduce memory leaks.
- Timeouts, subscriptions, event listeners, and other effects that are no longer needed should be disposed..

```
import { useState, useEffect } from 'react';
import { createConnection } from './chat.js';

function ChatRoom({ roomId }) {
  const [serverUrl, setServerUrl] = useState('https://localhost:12

  useEffect(() => {
    const connection = createConnection(serverUrl, roomId);
    connection.connect();
    return () => {
      connection.disconnect();
    };
  }, [serverUrl, roomId]);
```

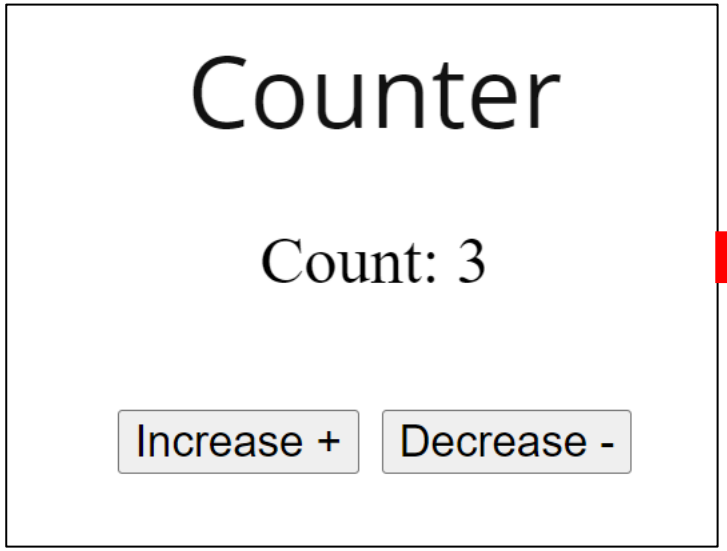
Cleanup

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const intervalId = setInterval(() => {
      setCount(count + 1); // You want to increment the counter every second...
    }, 1000)
    return () => clearInterval(intervalId);
  }, [count]); // 🚩 ... but specifying `count` as a dependency always resets the interval.
  // ...
}
```

useReducer

- An alternative to ***useState***
- ***useReducer*** is usually preferable to useState when you have complex state logic that involves multiple sub-values or when the next state depends on the previous one.
- ***useReducer*** also lets you optimize performance for components that trigger deep updates because you can pass dispatch down instead of callbacks.

Example




```
import Counter from "./components/Counter";

export default function Home() {
return (
        <div >
          <Counter/>
        </div>
  );
}
```

useReducer

- The ***useReducer*** hook
  - accept 2 arguments: the **reducer function** and the **initial state**.
  - returns an array of 2 items: the **current state** and the **dispatch** function.
- Familiar with Redux

Current state

Dispatch function

```
const [state, dispatch] = useReducer(reducer, initialState);
```

reducer function contains your custom state logic

Reducer function

- Handle the state update logic.
- Pure function that accepts 2 parameters:
  1. action object
  2. the current state
- Depending on the action object, the reducer function must update the state and **return the new state**
- Should be defined outside of any components.

Example

- ***Reducer*** function

```
const counterReducer = (state, action) => {
        switch (action.type) {
            case 'INCREMENT':
                return { count: state.count + 1 };
            case 'DECREMENT':
                return { count: state.count - 1 };
            default:
                return state;
        }
    };
```

New state

What is *dispatch* in React

- The ***dispatch*** function accepts an object that represents the type of action we want to execute when it is called.
- Basically, it sends the type of action to the ***reducer*** function to perform its job, which, of course, is updating the state.
- Think of ***dispatch*** as a messenger that delivers instructions (***actions***) to the state manager (***reducer***).

Example

```
const counterReducer = (state, action) => { ...See previous page... };
const initialState = { count: 0 };
const Counter = () => {

    const [state, dispatch] = useReducer(counterReducer, initialState);
    const handleIncrease = () => {
        dispatch({ type: 'INCREMENT' });
    };
    const handleDecrease = () => {
        dispatch({ type: 'DECREMENT' });
    };

    return (...See the next pages... );
}

export default Counter;
```

Reducer definition

Calling dispatcher function with action parameter

Example

```
const Counter = () => {
    ...


    return (
        <div>
            <h1>Counter</h1>
            <p>Count: {state.count}</p>
            <div>
                <button onClick={handleIncrease}>Increase +</button>
                <button onClick={handleDecrease}>Decrease -</button>
            </div>
        </div>
    );
}
export default Counter;
```

Useful Links about useReducer

- https://dmitripavlutin.com/react-usereducer/
- https://www.w3schools.com/react/react_usereducer.asp

How to avoid passing callbacks down?

- Most people don’t enjoy manually passing callbacks through every level of a component tree
- It can feel like a lot of “plumbing”

Context Hook

- React Context is a way to manage state globally.
- The Problem
  - State should be held by the highest parent component in the stack that requires access to the state.
  - To illustrate, we have many nested components. The component at the top and bottom of the stack need access to the state.
  - To do this without Context, we will need to pass the state as "props" through each nested component. This is called "prop drilling"

When do you need context?

- The main idea of using the context is to allow your components to access global data and re-render when that global data is changed.
- *useContext* is of course the React Hook way to consume React Context.
- It's a way to share data to any descendent component without having to manually pass the props through every intermediary component.
- Context solves the props drilling problem:
  - global state
  - theme
  - application configuration
  - authenticated username
  - user settings
  - preferred language
  - a collection of services

Example

```
import React, { createContext, useContext } from "react";
const Context = createContext();

const App = () => {
  return (
    <Context.Provider value={{ data: "Data from context!" }}>
      <Child />
    </Context.Provider>
  );
};
```

```
const Child3 = () => {
  const context = useContext(Context);
  return <div>{context.data}</div>;
};
const Child2 = () => {
  return <Child3/>;
};
const Child = () => {
  return <Child2/>;
};
```

Steps for using Context

1. Creating the context
2. Providing the context
3. Consuming the context

Step 1: Creating the context

```
import { createContext } from 'react';

export const MyContext = createContext()
```

Step 2: Providing the context

- *Context.Provider* component available on the context instance is used to provide the context to its child components, no matter how deep they are.
- To set the value of context use the value prop available on the *<Context.Provider value={value} />*

Step 2: Providing the context

```
import { MyContext } from './components/MyContext’;

function MyComponent() {

const contextObject={header:"Today's Deals"}
  return (
    <div>
      <MyContext.Provider value={contextObject}>
          <FilteredTable></FilteredTable>
      </MyContext.Provider>
    </div>
  );
}
export default MyComponent;
```

all the components that'd like later to consume the context have to be wrapped inside the provider component.

Step 3: Consuming the context

- Consuming the context can be performed in 2 ways:
  - The first way is to use the useContext(Context) React hook:

```
import { useContext } from 'react';
import { MyContext } from './MyContext’;
const Table = () => {
const contextObject = useContext(MyContext);
    return (
        <>
         <h2>{contextObject.header}</h2>
            <table> ...
            </table>
        </>
    );
}
export default Table;
```

Step 3: Consuming the context

- The second way is by using a render function supplied as a child to Context.Consumer special component available on the context instance:

```
import { MyContext } from './MyContext';

function MyComponent() {
  return (
    <MyContext.Consumer>
      {value => <span>{value}</span>}
    </MyContext.Consumer>
  );
}
```

Using Object in Context

- An object can be used in a Context

Context

```
const AppContext = createContext(
    {
        theme: "light",
        user: null,
        language: "en"
    });
```

Provider

```
<AppContext.Provider value={{
        theme: "light",
        user: null,
        language: "en"
    });}>
    <Child />
</AppContext.Provider>
```

Child component

```
const { theme, user, language } = useContext(AppContext);
```

Note => When the object changes (even if only one property changes), **all consuming components will re-render**.

References

- https://www.telerik.com/blogs/react-usecontext-hook
- https://dmitripavlutin.com/react-context-and-usecontext/
- https://www.geeksforgeeks.org/reactjs-usecontext-hook/
- https://daveceddia.com/usecontext-hook/
- https://react.dev/reference/react/useContext

Rules of Hooks

- Hooks are JavaScript functions, but they impose two additional rules:
  - Only call Hooks at the top level. Don't call Hooks inside loops, conditions, or nested functions.
    - https://reactjs.org/docs/hooks-rules.html#explanation
  - Only call Hooks from React function components. Don't call Hooks from regular JavaScript functions

Any Question?

Thank you for your attention

esma.meral@ozyegin.edu.tr